Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Матвиенко М.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 14.10.26

Москва, 2026

Постановка задачи

Вариант 15.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков.

Вариант: есть колода из 52 карт, рассчитать экспериментально (метод Монте-Карло) вероятность того, что сверху лежат две одинаковых карты. Количество раундов задаётся ключом программы

Общий метод и алгоритм решения

Использованные системные вызовы:

- **pthread_create()** — создаёт новый поток, который начинает выполнение заданной функции. В ядре реализуется через системный вызов clone(2).
- **pthread_join()** — блокирует вызывающий поток до завершения указанного потока. В ядре реализу()ется через futex(2).

Программа реализует параллельный метод Монте-Карло для оценки вероятности того, что первые две карты перемешанной колоды окажутся одинаковыми по номиналу или масти. Колода стандартная: 52 карты, 13 номиналов и 4 масти.

Количество потоков и количество раундов передаются в аргументах командной строки. Если аргументов меньше двух или значения меньше 1, программа выводит подсказку и завершается.

Раунды распределяются между потоками поровну: поток i получает интервал [i*N/T, (i+1)*N/T). Каждый поток строит собственную копию колоды и использует rand_r с сидом 1 + thread_id, так что у каждого получается независимая последовательность случайных чисел. Поскольку каждый поток работает только со своими данными, синхронизация не нужна — итоговый счётчик просто записывается в task->result.

Внутри потока каждый раунд выглядит так: колода перемешивается, затем проверяются первые две карты — если они совпали по номиналу или масти, локальный счётчик увеличивается.

После join всех потоков их результаты суммируются, и выводится итог: из N перемешиваний сколько раз первые две карты совпали. Для сравнения выводится и теоретическая вероятность 390/1326 ≈ 0,294: всего пар карт C(52,2) = 1326, из них 13 * C(4,2) = 78 совпадают по номиналу и 4 * C(13,2) = 312 — по масти.

Весь эксперимент прогоняется 25 раз подряд, время каждого прогона замеряется через clock_gettime(CLOCK_MONOTONIC), а после цикла выводится среднее время — так замеры получаются стабильнее, чем по одному запуску.

Число потоков	Время выполнения (мс)	Ускорение	Эффективность
1	6883	0,941	0,94116
2	5077	1,276	0,63798
4	2131	3,040	0,75997
8	2154	3,007	0,37593
16	1863	3,477	0,21732
32	1904	3,402	0,10632
128	2176	2,977	0,02326
1024	2201	2,943	0,00287
4096	2653	2,442	0,00060

Логических ядер 4, последовательная версия ушла в 6478 мс — по ней и считается ускорение, эффективность — это ускорение, делённое на число потоков.

При одном потоке время чуть хуже, чем у последовательной версии, — это накладные расходы на создание и ожидание потока. При двух ускорение заметно ниже идеала, на эту строку я бы не смотрел.

Основная картина начинается с четырёх потоков: ускорение около 3, эффективность 0,76 — почти линейное ускорение на четырёх ядрах, как и ожидалось. Дальше время перестаёт падать: ядер те же, а лишние потоки просто ждут планировщика. При 16 потоках время чуть меньше, чем при 4, но это разброс замеров, ускорение примерно то же.

На больших числах начинается деградация: при 128–4096 потоках время начинает расти, потому что создание тысяч потоков и переключения контекста стоят дороже, чем выигрывают.

Вывод: ускорение ограничено количеством ядер, максимум около 3,5 раз. Оптимально держать потоков сколько ядер, больше — уже ничего не выигрываешь.

Код программы

lab2_parallel.c

```
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>
#include <stdbool.h>


typedef struct {
    size_t thread_id;
    size_t start;       // начальный раунда (включительно)
    size_t end;         // конечный раунд (не включительно)
    uint64_t result;
} Task;


typedef struct {
    uint8_t rank; // номинал от 0 до 12
    uint8_t suit; // масть от 0 до 3
} Card;
```

```
void out_str(const char *s)
{
	size_t len = 0;
	while (s[len] != '\0')
		len++;
	write(STDOUT_FILENO, s, len);
}

void out_uint(int x)
{
	char buf[21];
	int pos = 20;

	while (x > 0)
   {
		buf[pos--] = (char)('0' + x % 10);
		x /= 10;
	}

	write(STDOUT_FILENO, buf + pos + 1, 20 - pos);
}

void deck_shuffle(Card *deck, unsigned *seed) // перемешивание колоды
{
	for (int i = 51; i > 0; i--) {
		int j = rand_r(seed) % (i + 1);
		Card tmp = deck[i];
		deck[i] = deck[j];
		deck[j] = tmp;
	}
```

```
}

bool cards_match(const Card *a, const Card *b)
{
	return a->rank == b->rank || a->suit == b->suit;
}

void *work(void *_args)
{
	Task *task = (Task *)_args;

	Card deck[52];
	int number = 0;
	for (int i = 0; i < 13; i++)
   {
		for (int j = 0; j < 4; j++)
	   {
			deck[number].rank = i;
			deck[number].suit = j;
			number++;
		}
   }
	unsigned seed = 1 + task->thread_id;
	uint64_t local = 0;

	for (int r = task->start; r < task->end; r++)
   {
		deck_shuffle(deck, &seed);
		if (cards_match(&deck[0], &deck[1]))
			local++;
```

```
        }
        task->result = local;
        return NULL;
}

int main(int argc, char** argv)
{
   if (argc != 3)
   {
      out_str("Используйте: ");
                out_str(argv[0]);
                out_str(" <количество потоков> <количество раундов>\n");
                return 1;
   }

   long t = strtol(argv[1], NULL, 10);
   long n = strtol(argv[2], NULL, 10);

   if (t < 1 || n < 1)
   {
                out_str("Число потоков и раундов должно быть >= 1\n");
                return 1;
        }

        size_t T = (size_t)t;
        size_t N = (size_t)n;

   uint64_t buf = 0;
   int maxp = 25;
   for (int p = 0; p < maxp; p++)
```

```
{
    struct timespec start, stop;  // таймер
    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_t *threads = malloc(T * sizeof(pthread_t)); // массив номеров
    Task *tasks = malloc(T * sizeof(Task)); // массив заданий
    if (!threads || !tasks)
        return 1;

    for (int i = 0; i < T; i++)
    {
        tasks[i].thread_id = i;
        tasks[i].start = i * N / T;
        tasks[i].end = (i + 1) * N / T;
        tasks[i].result = 0;
    }

    for (int i = 0; i < T; i++)
        pthread_create(&threads[i], NULL, work, &tasks[i]);

    for (int i = 0; i < T; i++)
        pthread_join(threads[i], NULL);

    int result = 0;
    for (int i = 0; i < T; i++)
    {
        result += tasks[i].result;
    }
    out_str("Из ");
    out_uint(n);
```

```
        out_str(" перемешивания(ий) колоды ");
        out_uint(result);
        out_str(" раз первые две карты оказались одинаковыми по масти или номиналу.\n");

        out_str("Теоретическая вероятность этого равна 390/1326 ~ 0,294.\n");

        clock_gettime(CLOCK_MONOTONIC, &stop);
        long long delta_ms = (long long)(stop.tv_sec - start.tv_sec) * 1000 + (long long)(stop.tv_nsec - start.tv_nsec) / 1000000;

        out_str("Время выполнения: ");
        out_uint(delta_ms);
        out_str(" милисекунд(ы)\n");
        buf += delta_ms;
    }
    out_str("Среднее время выполнения: ");
    out_uint(buf/ maxp);
    out_str(" милисекунд(ы)\n");
}
```

lab2_sequentially.c

```
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>
#include <stdbool.h>

typedef struct {
        uint8_t rank; // номинал от 0 до 12
        uint8_t suit; // масть от 0 до 3
```

```
} Card;
void out_str(const char *s)
{
	size_t len = 0;
	while (s[len] != '\0')
		len++;
	write(STDOUT_FILENO, s, len);
}
void out_uint(int x)
{
	char buf[21];
	int pos = 20;
	while (x > 0)
    {
		buf[pos--] = (char)('0' + x % 10);
		x /= 10;
	}
	write(STDOUT_FILENO, buf + pos + 1, 20 - pos);
}
void deck_shuffle(Card *deck, unsigned *seed) // перемешивание колоды
{
	for (int i = 51; i > 0; i--) {
		int j = rand_r(seed) % (i + 1);
		Card tmp = deck[i];
		deck[i] = deck[j];
		deck[j] = tmp;
	}
}
bool cards_match(const Card *a, const Card *b)
{
```

```
        return a->rank == b->rank || a->suit == b->suit;
}
int main(int argc, char** argv)
{
   if (argc != 2)
   {
      out_str("Используйте: ");
                out_str(argv[0]);
                out_str(" <количество раундов>\n");
                return 1;
   }
   long n = strtol(argv[1], NULL, 10);
   if (n < 1)
   {
                out_str("Число раундов должно быть >= 1\n");
                return 1;
        }
        uint64_t N = (uint64_t)n;
   struct timespec start, stop;  // таймер
        clock_gettime(CLOCK_MONOTONIC, &start);
   uint64_t result = 0;
        Card deck[52];
        int number = 0;
        for (int i = 0; i < 13; i++)
   {
                for (int j = 0; j < 4; j++)
      {
                        deck[number].rank = i;
                        deck[number].suit = j;
                        number++;
```

```
            }
    }
        unsigned seed = 1;
        for (int r = 0; r < n; r++)
    {
            deck_shuffle(deck, &seed);
            if (cards_match(&deck[0], &deck[1]))
                result++;
        }
    out_str("Из ");
    out_uint(n);
    out_str(" перемешивания(ий) колоды ");
    out_uint(result);
    out_str(" раз первые две карты оказались одинаковыми по масти или номиналу.\n");

    out_str("Теоретическая вероятность этого равна 390/1326 ~ 0,294.\n");

    clock_gettime(CLOCK_MONOTONIC, &stop);
        long long delta_ms = (long long)(stop.tv_sec - start.tv_sec) * 1000 + (long long)(stop.tv_nsec - start.tv_nsec) / 1000000;

        out_str("Время выполнения: ");
        out_uint(delta_ms);
        out_str(" милисекунд(ы)\n");
}
```

Протокол работы программы

Ввод программы: 4 потока, 20 000 000 раундов.

Каждый прогон выводит такие строки:

```
Из 20000000 перемешивания(ий) колоды 5884930 раз первые две карты оказались одинаковыми по масти или
номиналу.
Теоретическая вероятность этого равна 390/1326 ~ 0,294.
Время выполнения: 1718 милисекунд(ы)
Из 20000000 перемешивания(ий) колоды 5884930 раз первые две карты оказались одинаковыми по масти или
номиналу.
Теоретическая вероятность этого равна 390/1326 ~ 0,294.
Время выполнения: 1704 милисекунд(ы)
```

После этого идёт ещё 23 таких вывода. В конце показывается среднее время выполнения:

```
Среднее время выполнения: 1742 милисекунд(ы)
```

Вывод

Сделала программу с параметром параллельности и параметром количества потоков. Изначально думала, что это будет сложно реализовать какой-то именной метод экспериментального определения вероятности, но задача оказалось очень приятной. Не знала, что в Си нет библиотеки, позволяющей одним вызовом перемешать массив с какими то данными, только генераторы случайных чисел, поэтому написала сама простой метод с линейной сложностью.